



Java Learning Center

No.1 In Java Training & placement

Java Learning Center

java.util

Put into Knowledge

Doc Version - 1.0

Author

Srinivas Dande

~~Java~~ Kamlesh -



Introduction

- ♦ java.util is the built-in package available in Java language.
- ♦ This package contains the collections framework, legacy collection classes, event model, date and time facilities, internationalization, and miscellaneous utility classes.

Array and Legacy Classes

- ♦ Limitations with array:
 - Arrays are homogeneous data structure i.e only one type of elements it can store.
 - Arrays are static in nature i.e size cannot be modified once created.
 - Insert and delete operation in middle require more shifting of elements.
etc.
- ♦ Java vendor has given some classes and interfaces to solve the limitation of array before Java 2.
- ♦ These classes are called as legacy classes.
- ♦ These interfaces are called as legacy interfaces.
- ♦ Following are legacy classes & interfaces available:

Classes:

1. Stack
2. Vector
3. Dictionary
4. Hashtable
5. Properties

Interface:

1. Enumeration

- ♦ Collection of objects can be managed in two forms:
 - a) Without key and value format
Ex:
Collection of Emails
Collection of Phones
etc.
 - b) With key and value format
Ex:
Collection of Country and corresponding Capital.
Collection of ID and corresponding Name.
etc.
- ♦ Vector and Stack will be used to manage the collection of elements without key-value.
- ♦ Properties, Dictionary and Hastable are used to manage the element as key-value format. The key must be unique in the collection and can be used to identify the value.
- ♦ Enumeration interface is a legacy cursor that is used to access data from legacy classes one by one.

Java Learning Center

No.1 In Java Training & placement

Lab1236.java

```
import java.util.*;
public class Lab1236 {
    public static void main(String[] args) {
        Vector v= new Vector();
        v.addElement("Sri");
        v.addElement("Vas");
        v.addElement("Dande");
        System.out.println(v);
    }
}
```

Lab1237.java

```
import java.util.*;
public class Lab1237 {
    public static void main(String[] args) {
        Properties p= new Properties();
        p.put("JLC-99", "Sri");
        p.put("JLC-88", "Vas");
        p.put("JLC-77", "Dande");
        System.out.println(p);
    }
}
```

Lab1238.java

```
import java.util.*;
public class Lab1238{
    public static void main(String[] args) {
        Vector v= new Vector();
        System.out.println(v);
        System.out.println(v.size());
        System.out.println(v.capacity());
        v.addElement("Sri");
        v.addElement("Dande");
        v.addElement("Vas");
        v.addElement("Java");
    }
}
```

```
v.addElement("JSP");
v.addElement("EJB");
v.addElement("JPA");
System.out.println(v);
System.out.println(v.size());
System.out.println(v.capacity());
v.addElement("Database");
System.out.println(v);
System.out.println(v.size());
System.out.println(v.capacity());
}
```

Lab1239.java

```
import java.util.*;
public class Lab1239{
    public static void main(String[] args) {
        Vector v = new Vector();
        Student st = new Student(99);
        v.addElement(st);
        System.out.println(v);
        st.sid = 88;
        System.out.println(v);
    }
}
```

```
class Student{
    int sid;
    Student(int sid){
        this.sid = sid;
    }
    public String toString() {
        return ""+sid;
    }
}
```



- ♦ equals() with the object is used to identify the required object from collection.

Lab1240.java

```
import java.util.*;
public class Lab1240 {
    public static void main(String[] args) {
        Vector v = new Vector();
        Student st = new Student(99);
        v.addElement(st);
        System.out.println(v.contains(new Student(99)));
    }
    class Student{
        int sid;
        Student(int sid){
            this.sid = sid;
        }
        public String toString() {
            return ""+sid;
        }
        public boolean equals(Object obj) {
            System.out.println("** equals() **");
            return super.equals(obj);
        }
    }
}
```

- Compare to add

Lab1241.java

```
import java.util.*;
public class Lab1241 {
    public static void main(String[] args) {
        Vector v = new Vector();
        Student st = new Student(99);
        v.addElement(st);
        System.out.println(v.contains(new Student(99)));
    }
    class Student{
        int sid;
        Student(int sid){
            this.sid = sid;
        }
        public String toString() {
            return ""+sid;
        }
        public boolean equals(Object obj) {
            System.out.println("** equals() **");
            if(obj instanceof Student){
                Student st =(Student)obj;
                return this.sid == st.sid;
            }
            return false;
        }
    }
}
```

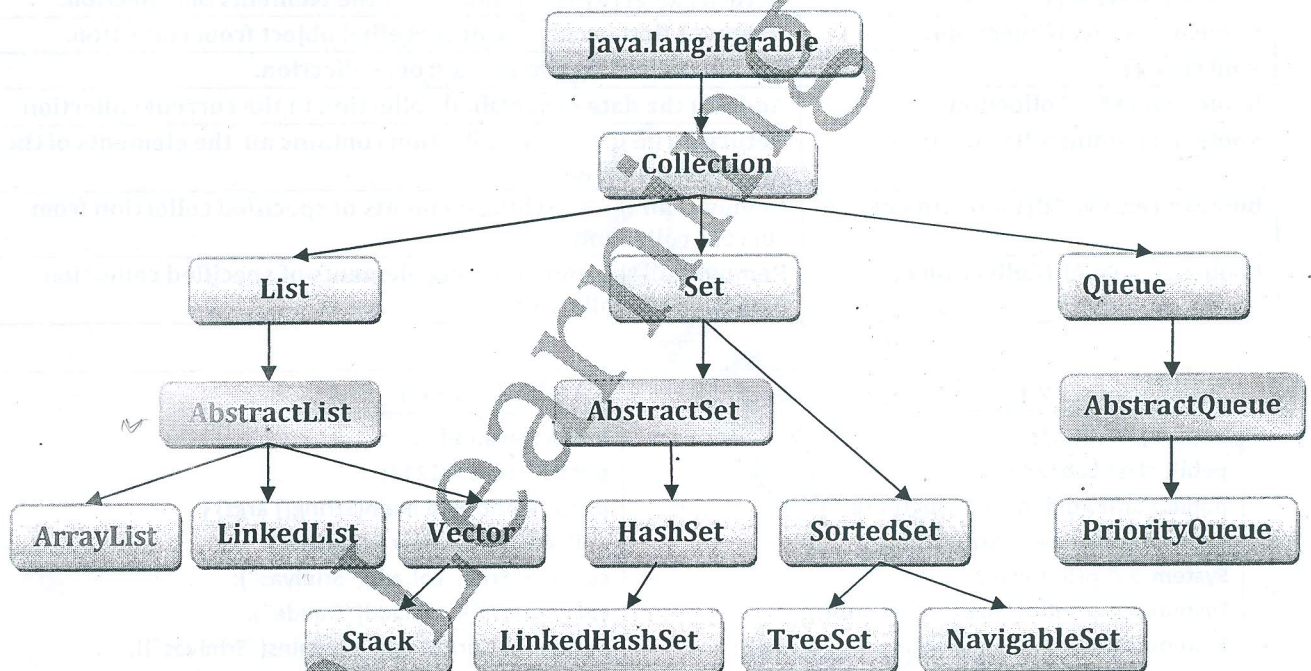
- Compare to contain

Collection Framework

- ♦ Limitations with legacy class:
 - There is no specific scenario specified that in which condition which classes will be used.
 - Almost all the methods defined in the legacy classes are synchronized. Because of these the same object cannot be accessed by multiple threads concurrently.
- ♦ Java Vendor has provided some new API (known as Collection Framework) from Java 2 to solve the problems of Legacy classes.
- ♦ Collection Framework provides an architecture to store and manipulate the group of objects.
- ♦ Operations like searching, sorting, insertion, deletion etc. can be performed on group of objects using Collection API.



- ♦ Collections Framework consists of following parts:
 - Interfaces
 - Implementation Classes
 - Algorithms (Methods to provide some common functionalities like searching, sorting and shuffling etc.)
- ♦ Following are the top level interfaces available in Collection framework:
 - Collection
 - Map
 - Iterator
- ♦ Collection subclasses is used to manage the collection of elements without key and value format.
- ♦ Map subclasses is used to manage the collection of elements with key and value format.
- ♦ Iterator will be used to access the data from the collection.



- ♦ Collection interface has following sub interfaces:
 - List
 - Set
 - Queue (from Java5)



Java Learning Center

No.1 In Java Training & placement

Interfaces	Abstract Classes	Concrete Sub classes	
Iterable	AbstractList	ArrayList	HashSet
Collection	AbstractSet	LinkedList	LinkedHashSet
List	AbstractQueue	Vector	TreeSet
Set		Stack	NavigableSet
SortedSet		PriorityQueue	
Queue			

Methods of Collection Interface	
Method	Description
<code>boolean add(Object obj)</code>	Adds objects to collection.
<code>int size()</code>	Returns no. of objects available in collection.
<code>boolean isEmpty()</code>	Checks wheather collection is empty or not.
<code>boolean contains(Object obj)</code>	Checks wheather collection has specified object or not.
<code>Iterator iterator()</code>	Returns Iterator subclass object.
<code>Object[] toArray()</code>	Returns an array containing all of the elements of collection.
<code>boolean remove(Object obj)</code>	Removes first occurance of specefied object from collection.
<code>void clear()</code>	Removes all of the elements from collection.
<code>boolean addAll(Collection c)</code>	Adds all the data of specified collection in the current collection
<code>boolean containsAll(Collection c)</code>	Returns true if current collection contains all the elements of the specified collection.
<code>boolean removeAll(Collection c)</code>	Removes all the matching elements of specified collection from current collection.
<code>boolean retainAll(Collection c)</code>	Removes all the non matching elements of specified collection from current collection.

Lab1242.java

```
import java.util.*;
public class Lab1242{
    public static void main(String[] args) {
        Collection col = new ArrayList();
        System.out.println(col);
        System.out.println("Size :" + col.size());
        System.out.println("Empty :" + col.isEmpty());
        System.out.println();
        col.add("Sri"); col.add("Nivas"); col.add("Srini");
        col.add("SD"); col.add("Dande");
        System.out.println(col);
        System.out.println("Size :" + col.size());
        System.out.println("Empty :" + col.isEmpty());
    }
}
```

Lab1243.java

```
import java.util.*;
public class Lab1243{
    public static void main(String[] args) {
        Collection col = new ArrayList();
        col.add("Sri"); col.add("Srinivas");
        col.add("SD"); col.add("Dande");
        System.out.println(col.contains("Srinivas"));
        System.out.println(col.contains("Java"));
        System.out.println(col);
        System.out.println(col.remove("JDBC"));
        System.out.println(col.remove("Srinivas"));
        System.out.println(col);
        col.clear();
        System.out.println(col);
        System.out.println(col.size());
    }
}
```



Java Learning Center

No.1 In Java Training & placement

Lab1244.java

```
import java.util.*;
public class Lab1244{
    public static void main(String[] args) {
        Collection col = new ArrayList();
        col.add("Sri"); col.add("Srinivas"); col.add("Dande");
        Object arr[]=col.toArray();
        for (int i = 0; i < arr.length; i++) {
            System.out.println(arr[i]);
            arr[i]="JLC";
        }
        System.out.println(col);
    }
}
```

Lab1245.java

```
import java.util.*;
public class Lab1245{
    public static void main(String[] args) {
        Collection col = new ArrayList();
        col.add("Sri"); col.add("Srinivas"); col.add("Dande");
        Iterator it=col.iterator();
        while(it.hasNext()){
            Object obj=it.next(); System.out.println(obj);
            if(obj.equals("SD")) it.remove();
        }
        System.out.println(col);
    }
}
```

After class see Notebook
check where next element is about to be removed

Lab1246.java

```
import java.util.*;
public class Lab1246{
    public static void main(String[] args) {
        Collection col = new ArrayList();
        col.add("Sri"); col.add("Srinivas"); col.add("Dande");
        System.out.println(col + "\t" + col.size());
        Collection col2=new ArrayList();
        col2.add("Java"); col2.add("JDBC"); col2.add("JSP");
        System.out.println(col2 + "\t" + col2.size());
        col.add(col2);
        System.out.println(col + "\t" + col.size());
    }
}
```

Lab1247.java

```
import java.util.*;
public class Lab1247{
    public static void main(String[] args) {
        Collection col = new ArrayList();
        col.add("Sri"); col.add("Srinivas"); col.add("Dande");
        System.out.println(col + "\t" + col.size());
        Collection col2=new ArrayList();
        col2.add("Java"); col2.add("JDBC"); col2.add("JSP");
        System.out.println(col2 + "\t" + col2.size());
        col.addAll(col2);
        System.out.println(col + "\t" + col.size());
    }
}
```

3
6

Lab1248.java

```
import java.util.*;
public class Lab1248{
    public static void main(String[] args) {
        Collection col = new ArrayList();
        col.add("JDBC"); col.add("JSP"); col.add("Srinivas");
        col.add("Java"); col.add("Dande");
        Collection col2=new ArrayList();
        col2.add("Java"); col2.add("JDBC"); col2.add("JSP");
        Collection col3=new ArrayList();
        col3.add("Java"); col3.add("JDBC"); col3.add("EJB");
        System.out.println(col + "\n" + col2 + "\n" + col3);
        System.out.println(col.containsAll(col2));
        System.out.println(col.containsAll(col3));
    }
}
```

Lab1249.java

```
import java.util.*;
public class Lab1249{
    public static void main(String[] args) {
        Collection col = new ArrayList();
        col.add("JDBC"); col.add("Sri"); col.add("JSP");
        col.add("Srinivas"); col.add("Java"); col.add("Dande");
        Collection col2=new ArrayList();
        col2.add("Java"); col2.add("JDBC"); col2.add("JSP");
        System.out.println(col); System.out.println(col2);
        System.out.println(col.removeAll(col2));
        System.out.println(col); System.out.println(col2);
    }
}
```

matching element will remove



Lab1250.java

```
import java.util.*;
public class Lab1250 {
    public static void main(String[] args) {
        Collection col = new ArrayList();
        col.add("JDBC"); col.add("Sri"); col.add("JSP");
        col.add("Srinivas"); col.add("Java");
        col.add("Dande");
    }
}
```

```
Collection col2=new ArrayList();
col2.add("Java"); col2.add("JDBC"); col2.add("JSP");
System.out.println(col);
System.out.println(col2);
System.out.println(col.retainAll(col2));
System.out.println(col);
System.out.println(col2);
}}
```

machine element will remove

List and their Subclasses

- List is an interface available in java.util package and it is extending java.util.Collection interface.
- List subclasses allow duplicate elements.

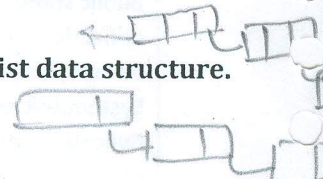
ArrayList

- ArrayList allows duplicates and it is implemented as a resizable array. This is one of the most widely used concrete class.
- Fast to search, but slow to insert or delete.



LinkedList

- LinkedList allows duplicates and internally implements a doubly-linked list data structure.
- Fast to insert or delete elements, but slow for searching elements.



Vector

- Vector methods are synchronized, so vector object can't be accessed by multiple threads at a time.
- Vector elements will be stored internally using indexing notation.

METHODS OF LIST INTERFACE	
Method	Description
Object get(int idx)	Returns the element available at specified index.
Object remove(int idx)	Removes the element available at specified index.
void add(int idx, Object obj)	Add/Insert the element at the specific index
int indexOf(Object obj)	Returns the first position of specified object.
int lastIndexOf(Object obj)	Returns the last position of specified object.
boolean addAll(int idx, Collection col)	Adds all the element of collection at specified index.
List subList(int stIdx, int endIdx)	Returns portion of list.
ListIterator listIterator()	Returns ListIterator object that points before to first record.
ListIterator listIterator(int stIdx)	Returns ListIterator object that points to specified index.
Object set(int idx, Object obj)	Replaces the object available at specified index.



Java Learning Center

No.1 In Java Training & placement

Lab1251.java

```
import java.util.*;
public class Lab1251 {
    public static void main(String[] args) {
        List list = new ArrayList();
        list.add("Sri"); list.add("Nivas"); list.add("Dande");
        System.out.println(list);
        list.add(1, "Java");
        System.out.println(list);
        list.remove(2);
        System.out.println(list);
    }
}
```

Lab1252.java

```
import java.util.*;
public class Lab1252 {
    public static void main(String[] args) {
        List list = new ArrayList();
        list.add("Sri"); list.add("Nivas"); list.add("Dande");
        System.out.println(list);
        list.set(1, "Java");
        System.out.println(list);
        Object obj=list.get(2);
        System.out.println(obj);
    }
}
```

Lab1253.java

```
import java.util.*;
public class Lab1253 {
    public static void main(String[] args) {
        List list = new ArrayList();
        list.add("Sri"); list.add("Nivas"); list.add("Dande");
        list.add("Java"); list.add("Nivas"); list.add("JDBC");
        System.out.println(list);
        int x=list.indexOf("Nivas");
        System.out.println(x);
        int y=list.lastIndexOf("Nivas");
        System.out.println(y);
    }
}
```

Lab1254.java

```
import java.util.*;
public class Lab1254 {
    public static void main(String[] args) {
        List list = new ArrayList();
        list.add("Sri"); list.add("Nivas");
        list.add("Dande"); list.add("Java");
        list.add("Nivas"); list.add("JDBC");
        List list2=list.subList(1, 3);
        System.out.println(list);
        System.out.println(list2);
    }
}
```

Iterator interface

- ♦ Iterator is an interface available in java.util package.
- ♦ It is a cursor to access the elements from collection subclasses one by one.
- ♦ iterator() method can be invoked with collection subclass object to get the object of Iterator.
- ♦ Inner class is defined in collection subclass classes that returns the object of Iterator subclass.
- ♦ Iterator interface contains following methods:
 1. boolean hasNext():
Checks whether next element is available or not.
 2. Object next() :
Moves the pointer and returns elements from collection.
 3. void remove():
Removes the current elements from Collections.

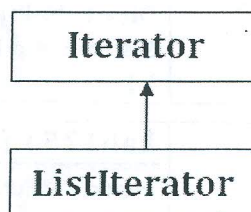
Limitation

*only remove
only forward reference.
Not use with Set and queue*



ListIterator interface

- ♦ ListIterator is an interface which is extending Iterator interface.
- ♦ ListIterator provides some extra methods that can be used to access or to manage the data for the List subclasses.
- ♦ ListIterator methods cannot be used with other subclasses of Collection.
- ♦ By using ListIterator you can :
 - Access the index of elements.
 - Access the elements in both forward and reverse direction.
 - Add, Remove and Replace the data of List subclasses.
 - Access the elements starting from the specified index.



Lab1255.java

```
import java.util.*;
public class Lab1255{
public static void main(String[] args) {
List list = new ArrayList();
list.add("Sri"); list.add("Nivas"); list.add("Dande");
list.add("Java"); list.add("JDBC"); list.add("JSP");
Iterator it = list.iterator();
while(it.hasNext()){
Object obj=it.next();
System.out.println(obj);
}
}}
```

Lab1256.java

```
import java.util.*;
public class Lab1256 {
public static void main(String[] args) {
List list = new ArrayList();
list.add("Sri"); list.add("Nivas"); list.add("Dande");
list.add("Java"); list.add("JDBC"); list.add("JSP");
ListIterator lit = list.listIterator();
while(lit.hasNext()){
Object obj=lit.next();
System.out.println(obj);
}
}}
```

Lab1257.java

```
import java.util.*;
public class Lab1257{
public static void main(String[] args) {
List list = new ArrayList();
list.add("Sri"); list.add("Nivas"); list.add("Dande");
list.add("Java"); list.add("JDBC"); list.add("JSP");
ListIterator lit=list.listIterator();
System.out.println(list);
}
```

```
while(lit.hasNext()){
Object obj = lit.next();
System.out.println(obj);
if(obj.equals("Nivas")) lit.remove();
else if(obj.equals("Java")) lit.set("Core Java");
else if(obj.equals("JSP")) lit.add("Servlets");
}
System.out.println(list);
}
```



Java Learning Center

No.1 In Java Training & placement

Lab1258.java

```
import java.util.*;
public class Lab1258 {
    public static void main(String[] args) {
        List list = new ArrayList();
        list.add("Sri"); list.add("Nivas"); list.add("Dande");
        list.add("Java"); list.add("JDBC"); list.add("JSP");
        ListIterator lit = list.listIterator();
        System.out.println(" FORWARD ORDER");
        while(lit.hasNext()){
            int indx=lit.nextIndex();
            Object obj=lit.next();
            System.out.println(indx+"\t"+obj);
        }
        System.out.println(" REVERSE ORDER");
        while(lit.hasPrevious()){
            int indx=lit.previousIndex();
            Object obj=lit.previous();
            System.out.println(indx+"\t"+obj);
        }
    }
}
```

Lab1259.java

```
import java.util.*;
public class Lab1259{
    public static void main(String[] args) {
        List list = new ArrayList();
        list.add("Sri"); list.add("Nivas"); list.add("Dande");
        list.add("Java"); list.add("JDBC"); list.add("JSP");
        ListIterator lit = list.listIterator(2);
        do{
            int indx=lit.nextIndex();
            Object obj=lit.next();
            System.out.println(indx+"\t"+obj);
        }while(lit.hasNext());
        System.out.println();
        lit=list.listIterator(5);
        do{
            int indx=lit.previousIndex();
            Object obj=lit.previous();
            System.out.println(indx+"\t"+obj);
        }while(lit.hasPrevious());
    }
}
```

Set interface

- ♦ Set is an interface available in java.util package and it is extending java.util.Collection interface.
- ♦ Set subclasses does not allow duplicate elements like List.
- ♦ hashCode() and equal() method will be used internally to identify the unique object in Set, i.e if two object have the same hash code value and the equals() method is also returning true then that object will be considered as duplicate object.
- ♦ Set interface doesn't have any extra methods.
- ♦ All the methods of Collection interface can be used with Set subclasses.

HashSet

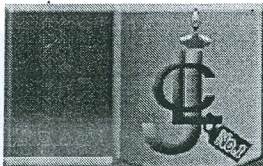
- ♦ Fast for searching and retrieving elements.
- ♦ It does not maintain any order for stored elements.

→ Store the data in the Random order

LinkedHashSet

- ♦ LinkedHashSet is subclass of HashSet.
- ♦ It stores the data in the order as added by the user.

→ Store the data in the order as you are inserting.



Java Learning Center

No.1 In Java Training & placement

TreeSet

← Store in sorted order.

- ♦ It stores the elements in a sorted order.
- ♦ `compareTo()` method will be used to identify the object uniquely and to manage the order of elements in `TreeSet`.
- ♦ The object you are adding in the `TreeSet` must be the subtype of `Comparable` interface.
- ♦ `TreeSet` allows to store elements of similar type.
- ♦ Null value cannot be stored in `TreeSet`.

you object is comparable or not

— Same type value only store

NavigableSet

- ♦ `NavigableSet` is an interface added in Java 6.
- ♦ It is subtype of `SortedSet`.
- ♦ Its functionality is almost similar to `TreeSet` except it provides some extra methods for easy Navigation.
- ♦ Elements from `NavigableSet` can be accessed in both forward and reverse order.

Lab1260.java

```
import java.util.*;
public class Lab1260 {
    public static void main(String[] args) {
        HashSet set = new HashSet();
        set.add("Sri");
        set.add("Nivas");
        set.add(99);
        set.add("Dande");
        set.add("JLCINDIA");
        System.out.println(set);
    }
}
```

Random order

Lab1261.java

```
import java.util.*;
public class Lab1261 {
    public static void main(String[] args) {
        LinkedHashSet set = new LinkedHashSet();
        set.add("Sri");
        set.add("Nivas");
        set.add(99);
        set.add("Dande");
        set.add("JLCINDIA");
        System.out.println(set);
    }
}
```

Same order

Lab1262.java

```
import java.util.*;
public class Lab1262 {
    public static void main(String[] args) {
        TreeSet set = new TreeSet();
        set.add("Sri");
        set.add("Nivas");
        set.add("Dande");
        set.add("JLCINDIA");
        System.out.println(set);
    }
}
```

Sorted order

Lab1263.java

```
import java.util.*;
public class Lab1263 {
    public static void main(String[] args) {
        TreeSet set = new TreeSet();
        set.add("Sri");
        set.add("Nivas");
        set.add("JLCINDIA");
        set.add(99);
        set.add("Dande");
        System.out.println(set);
    }
}
```

Class cast Exception

R.000

TreeSet

- It stores the elements in a sorted order.
- compareTo() method will be used to identify the object uniquely and to manage the order of elements in TreeSet.
- The object you are adding in the TreeSet must be the subtype of Comparable interface.
- TreeSet allows to store elements of similar type.
- Null value cannot be stored in TreeSet.

NavigableSet

- NavigableSet is an interface added in Java 6.
- It is subtype of SortedSet.
- Its functionality is almost similar to TreeSet except it provides some extra methods for easy Navigation.
- Elements from NavigableSet can be accessed in both forward and reverse order.

Lab1260.java

```
import java.util.*;
public class Lab1260 {
    public static void main(String[] args) {
        HashSet set = new HashSet();
        set.add("Sri");
        set.add("Nivas");
        set.add(99);
        set.add("Dande");
        set.add("JLCINDIA");
        System.out.println(set);
    }
}
```

Lab1261.java

```
import java.util.*;
public class Lab1261 {
    public static void main(String[] args) {
        LinkedHashSet set = new LinkedHashSet();
        set.add("Sri");
        set.add("Nivas");
        set.add(99);
        set.add("Dande");
        set.add("JLCINDIA");
        System.out.println(set);
    }
}
```

Lab1262.java

```
import java.util.*;
public class Lab1262 {
    public static void main(String[] args) {
        TreeSet set = new TreeSet();
        set.add("Sri");
        set.add("Nivas");
        set.add("Dande");
        set.add("JLCINDIA");
        System.out.println(set);
    }
}
```

Lab1263.java

```
import java.util.*;
public class Lab1263 {
    public static void main(String[] args) {
        TreeSet set = new TreeSet();
        set.add("Sri");
        set.add("Nivas");
        set.add("JLCINDIA");
        set.add(99);
        set.add("Dande");
        System.out.println(set);
    }
}
```



Lab1264.java

```
import java.util.*;
public class Lab1264{
    public static void main(String[] args) {
        LinkedHashSet set = new LinkedHashSet();
        set.add(new Long(65));
        set.add(new Byte((byte)65));
        set.add(new Integer(65));
        set.add("A");
        System.out.println(set);
    }
}
```

*all are store
equal method returns false*

Lab1265.java

```
import java.util.*;
public class Lab1265 {
    public static void main(String[] args) {
        LinkedHashSet set = new LinkedHashSet();
        set.add(new Student(99)); set.add(new Student(99));
        set.add(new Student(99)); set.add(new Student(99));
        System.out.println(set);
    }
    class Student{
        int sid;
        Student(int sid){
            this.sid = sid;
        }
        public String toString() {
            return ""+sid;
        }
    }
}
```

Lab1266.java

```
import java.util.*;
public class Lab1266 {
    public static void main(String[] args) {
        LinkedHashSet set = new LinkedHashSet();
        set.add(new Student(99));
        set.add(new Student(99));
        set.add(new Student(99));
        set.add(new Student(99));
        System.out.println(set);
    }
}
```

only one object added

```
class Student{
    int sid;
    Student(int sid){
        this.sid = sid;
    }
    public String toString() {
        return ""+sid;
    }
    public int hashCode() {
        System.out.println("*** hashCode() ***");
        return sid;
    }
    public boolean equals(Object obj) {
        System.out.println("*** equals() ***");
        if(obj instanceof Student){
            Student st = (Student)obj;
            return this.sid ==st.sid;
        }
        return false;
    }
}
```



Methods from NavigableSet	
E ceiling(E e)	Returns the least element in this set greater than or equal to the given element.
Iterator descendingIterator()	Returns an iterator over the elements in this set, in descending order.
NavigableSet descendingSet()	Returns a reverse order view of the elements contained in this set.
E floor(E e)	Returns the greatest element in this set less than or equal to the given element.
SortedSet<E> headSet(E toElement)	Returns a view of the portion of this set whose elements are strictly less than toElement.
NavigableSet<E> headSet(E toElement, boolean inclusive)	Returns a view of the portion of this set whose elements are less than (or equal to, if inclusive is true) toElement.
E higher(E e)	Returns the least element in this set strictly greater than the given element, or null if there is no such element.
Iterator iterator()	Returns an iterator over the elements in this set, in ascending order.
E lower(E e)	Returns the greatest element in this set strictly less than the given element.
E pollFirst()	Retrieves and removes the first (lowest) element, or returns null if this set is empty.
E pollLast()	Retrieves and removes the last (highest) element, or returns null if this set is empty.
NavigableSet subSet(E fromElement, boolean fromInclusive, E toElement, boolean toInclusive)	Returns a view of the portion of this set whose elements range from fromElement to toElement.
SortedSet subSet(E fromElement, E toElement)	Returns a view of the portion of this set whose elements range from fromElement, inclusive, to toElement, exclusive.
SortedSet tailSet(E fromElement)	Returns a view of the portion of this set whose elements are greater than or equal to fromElement.
NavigableSet tailSet(E fromElement, boolean inclusive)	Returns a view of the portion of this set whose elements are greater than (or equal to, if inclusive is true) fromElement.



Lab1267.java

```
import java.util.*;
public class Lab1267 {
    public static void main(String[] args) {
        TreeSet ts= new TreeSet();
        ts.add(10);      ts.add(5);
        ts.add(7);      ts.add(11);
        ts.add(3);      ts.add(9);
        ts.add(4);      ts.add(20);
        System.out.println(ts);
        Set ts1=ts.descendingSet();
        System.out.println(ts1);
        System.out.println("--Ascending Iterator--");
        Iterator it = ts.iterator();
        while(it.hasNext())
            System.out.println(it.next());
        System.out.println("Descending Iterator");
        Iterator it1 = ts.descendingIterator();
        while(it1.hasNext())
            System.out.println(it1.next());
    }
}
```

Lab1268.java

```
import java.util.*;
public class Lab1268 {
    public static void main(String[] args) {
        TreeSet ts= new TreeSet();
        //ts.add(...) same as previous lab
        System.out.println(ts);
        System.out.println("--subset()--");
        System.out.println(ts.subSet(4, 11));
        System.out.println(ts.subSet(4,false, 11,true));
        System.out.println(ts.subSet(4,true, 11,true));
        System.out.println("--headset()--");
        System.out.println(ts.headSet(9));
        System.out.println(ts.headSet(9,true));
        System.out.println(ts.headSet(2));
        System.out.println("--tailset()--");
        System.out.println(ts.tailSet(9));
        System.out.println(ts.tailSet(9,false));
    }
}
```

Lab1269.java

```
import java.util.*;
public class Lab1269 {
    public static void main(String[] args) {
        TreeSet ts= new TreeSet();
        //ts.add(...) same as previous lab

        System.out.println("--pollFirst/Last--");
        System.out.println(ts);
        System.out.println(ts.pollFirst());
        System.out.println(ts);
        System.out.println(ts);
        System.out.println(ts.pollLast());
        System.out.println(ts);

        System.out.println("***lower() < ***");
        System.out.println(ts.lower(8));
        System.out.println(ts.lower(50));
        System.out.println(ts.lower(3));
        System.out.println(ts.lower(4));
    }
}
```

```
System.out.println("***higher() > ***");
System.out.println(ts.higher(8));
System.out.println(ts.higher(50));
System.out.println(ts.higher(3));
System.out.println(ts.higher(9));

System.out.println("***ceiling() >= ***");
System.out.println(ts.ceiling(8));
System.out.println(ts.ceiling(10));
System.out.println(ts.ceiling(11));
System.out.println(ts.ceiling(12));

System.out.println("***floor() <= ***");
System.out.println(ts.floor(8));
System.out.println(ts.floor(10));
System.out.println(ts.floor(11));
System.out.println(ts.floor(12));
System.out.println(ts.floor(3));
}
}
```



java.lang.Comparable interface

- ♦ It is an interface of java.lang package that will be used to specify that the object of the class can be compared or not. If you have the requirement to compare the object for sorting purpose or to identify the higher and lower object then Java vendor has provided Comparable Interface.
- ♦ The following abstract method available in the comparable interface
 - public int compareTo(object obj)
- ♦ The object you want to compare must be the subtype of Comparable interface and must have to provide the implementation for the compareTo() method.
- ♦ If object will not be subtype of comparable then it will throw an exception at runtime
java.lang.ClassCastException

Lab1270.java

```
import java.util.*;
public class Lab1270 {
    public static void main(String[] args) {
        TreeSet ts = new TreeSet();
        ts.add("Sri");
        ts.add("Nivas");
        ts.add("Dande");
        ts.add("Abhi");
        ts.add("Rahul");
    }
}
```

S.O.P (Set)

Lab1271.java

```
import java.util.*;
public class Lab1271{
    public static void main(String[] args) {
        TreeSet ts = new TreeSet();
        ts.add(new Employee());
        ts.add(new Employee());
        ts.add(new Employee());
        ts.add(new Employee());
    }
}

class Employee { }
```

R. Err

Lab1272.java

```
import java.util.*;
public class Lab1272 {
    public static void main(String[] args) {
        System.out.println("** STUDENT INFO **");
        TreeSet set = new TreeSet();
        set.add(new Student(87, "Sri"));
        set.add(new Student(99, "Dande"));
        set.add(new Student(12, "SD"));
        set.add(new Student(45, "Nivas"));
        Iterator it = set.iterator();
        while (it.hasNext())
            System.out.println(it.next());
    }
}
```

```
class Student implements Comparable {
    int sid; String name;
    Student(int sid, String name) {
        this.sid = sid; this.name = name;
    }
    public int compareTo(Object obj) {
        if (obj instanceof Student) {
            Student st = (Student) obj;
            return this.sid - st.sid;
        }
        return 0;
    }
    public String toString() {
        return sid + "\t" + name;
    }
}
```



Java Learning Center

No.1 In Java Training & placement

Lab1273.java

```
import java.util.*;
public class Lab1273 {
    public static void main(String[] args) {
        System.out.println("\n\n** EMPLOYEE INFO **");
        TreeSet set = new TreeSet();
        set.add(new Employee(87, "Sri"));
        set.add(new Employee(99, "Dande"));
        set.add(new Employee(12, "SD"));
        set.add(new Employee(45, "Nivas"));
        Iterator it = set.iterator();
        while (it.hasNext()) {
            System.out.println(it.next());
        }
    }
}
```

*Ascending order name
will print*

```
class Employee implements Comparable {
    int eid;
    String name;
    Employee(int eid, String name) {
        this.eid = eid;
        this.name = name;
    }
    public int compareTo(Object obj) {
        if (obj instanceof Employee) {
            Employee em = (Employee) obj;
            return this.name.compareTo(em.name);
        }
        return 0;
    }
    public String toString() {
        return eid + "\t" + name;
    }
}
```

If you have some requirement where you are using the class implemented by other user and you can't implement Comparable interface.

Q) How can we store the StringBuffer object in TreeSet ?

Q) How can we store the String object in TreeSet but the order should be in descending order?